

## Module 2 - Data Structures and Algorithms

### 1. Array Traversal

```
Main.java
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         int arr[] = {10, 20, 30, 40, 50};
5
6         for (int i = 0; i < arr.length; i++) {
7             System.out.print(arr[i] + " ");
8         }
9     }
10 }
```

input

10 20 30 40 50

### 2. Linear Search

```
Main.java
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         int[] arr = {10, 25, 30, 45, 50};
5         int key = 45;
6
7         for(int i=0;i<arr.length;i++){
8             if(arr[i]==key){
9                 System.out.println("Found at index "+i);
10                return;
11            }
12        }
13
14        System.out.println("Not Found");
15    }
16 }
```

input

Found at index 3

### 3. Binary Search

```
Main.java
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         int[] arr={10,20,30,40,50,60};
5
6         int key=40;
7
8         int low=0,high=arr.length-1;
9
10        while(low<=high){
11            int mid=(low+high)/2;
12
13            if(arr[mid]==key){
14                System.out.println("Found at "+mid);
15                return;
16            }
17
18            else if(arr[mid]<key)
19                low=mid+1;
20
21            else
22                high=mid-1;
23        }
24
25        System.out.println("Not Found");
26    }
27 }
```

Found at 3

### 4. Bubble Sort

```
Main.java
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4
5         int arr[]={5,2,8,1,9};
6
7         for(int i=0;i<arr.length-1;i++){
8
9             for(int j=0;j<arr.length-i-1;j++){
10
11                 if(arr[j]>arr[j+1]){
12
13                     int temp=arr[j];
14                     arr[j]=arr[j+1];
15                     arr[j+1]=temp;
16                 }
17             }
18         }
19
20         System.out.println(Arrays.toString(arr));
21     }
22 }
23 }
```

[1, 2, 5, 8, 9]

## 5. Insertion sort

```
Main.java
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         int arr[]={8,5,2,9,1};
5         for(int i=1;i<arr.length;i++){
6             int key=arr[i];
7             int j=i-1;
8             while(j>=0 && arr[j]>key){
9
10                arr[j+1]=arr[j];
11                j--;
12            }
13            arr[j+1]=key;
14        }
15        System.out.println(Arrays.toString(arr));
16    }
17 }
```

input

[1, 2, 5, 8, 9]

## 6. Heap Sort

```
Main.java
1 import java.util.*;
2 public class Main {
3     static void heapify(int arr[],int n,int i){
4         int largest=i;
5         int left=2*i+1;
6         int right=2*i+2;
7         if(left<n && arr[left]>arr[largest])
8             largest=left;
9         if(right<n && arr[right]>arr[largest])
10            largest=right;
11        if(largest!=i){
12            int temp=arr[i];
13            arr[i]=arr[largest];
14            arr[largest]=temp;
15            heapify(arr,n,largest);
16        }
17    }
18    static void heapSort(int arr[]){
19        int n=arr.length;
20        for(int i=n/2-1;i>=0;i--){
21            heapify(arr,n,i);
22        }
23        for(int i=n-1;i>0;i--){
24            int temp=arr[0];
25            arr[0]=arr[i];
26            arr[i]=temp;
27            heapify(arr,i,0);
28        }
29    }
30    public static void main(String[] args){
31        int arr[]={12,11,13,5,6,7};
32        heapSort(arr);
33        System.out.println(Arrays.toString(arr));
34    }
35 }
```

[5, 6, 7, 11, 12, 13]

## 7. Merge Sort

```
Main.java
1 import java.util.*;
2 public class Main {
3     static void mergeSort(int arr[],int l,int r){
4         if(l<r){
5             int mid=(l+r)/2;
6             mergeSort(arr,l,mid);
7             mergeSort(arr,mid+1,r);
8             merge(arr,l,mid,r);
9         }
10    }
11
12    static void merge(int arr[],int l,int m,int r){
13        int n1=m-l+1;
14        int n2=r-m;
15        int L[]=new int[n1];
16        int R[]=new int[n2];
17        for(int i=0;i<n1;i++)
18            L[i]=arr[l+i];
19        for(int j=0;j<n2;j++)
20            R[j]=arr[m+1+j];
21        int i=0,j=0,k=l;
22        while(i<n1 && j<n2){
23            if(L[i]<=R[j])
24                arr[k++]=L[i++];
25            else
26                arr[k++]=R[j++];
27        }
28        while(i<n1)
29            arr[k++]=L[i++];
30        while(j<n2)
31            arr[k++]=R[j++];
32    }
33
34    public static void main(String args[]){
35        int arr[]={5,4,3,2,1};
36        mergeSort(arr,0,arr.length-1);
37        System.out.println(Arrays.toString(arr));
38    }
39 }
```

input

[1, 2, 3, 4, 5]

## 8. Quick Sort

```
Main.java
1 import java.util.*;
2 public class Main {
3     static int partition(int arr[],int low,int high){
4         int pivot=arr[high];
5         int i=low-1;
6         for(int j=low;j<high;j++){
7             if(arr[j]<pivot){
8                 i++;
9                 int temp=arr[i];
10                arr[i]=arr[j];
11                arr[j]=temp;
12            }
13        }
14        int temp=arr[i+1];
15        arr[i+1]=arr[high];
16        arr[high]=temp;
17        return i+1;
18    }
19    static void quickSort(int arr[],int low,int high){
20        if(low<high){
21            int pi=partition(arr,low,high);
22            quickSort(arr,low,pi-1);
23            quickSort(arr,pi+1,high);
24        }
25    }
26
27    public static void main(String args[]){
28        int arr[]={10,7,8,9,1,5};
29        quickSort(arr,0,arr.length-1);
30        System.out.println(Arrays.toString(arr));
31    }
32 }
```

input  
[1, 5, 7, 8, 9, 10]

## 9. Singly Linked List (Insert at End + Display)

```
Main.java  ⋮
1  import java.util.*;
2  class Node{
3      int data;
4      Node next;
5      Node(int data){
6          this.data=data;
7      }
8  }
9  class Main{
10     Node head;
11     void insert(int data){
12         Node newNode=new Node(data);
13         if(head==null){
14             head=newNode;
15             return;
16         }
17         Node temp=head;
18         while(temp.next!=null)
19             temp=temp.next;
20         temp.next=newNode;
21     }
22     void display(){
23         Node temp=head;
24         while(temp!=null){
25             System.out.print(temp.data+" ");
26             temp=temp.next;
27         }
28     }
29     public static void main(String[] args){
30         Main list=new Main();
31         list.insert(10);
32         list.insert(20);
33         list.insert(30);
34         list.display();
35     }
36 }
```

input

10 20 30

## 10. Insert at Beginning

```
Main.java
1 import java.util.*;
2 class Node{
3     int data;
4     Node next;
5     Node(int data){
6         this.data=data;
7     }
8 }
9 class Main{
10     Node head;
11     void insertBeginning(int data){
12         Node newNode=new Node(data);
13         newNode.next=head;
14         head=newNode;
15     }
16     void display(){
17         Node temp=head;
18         while(temp!=null){
19             System.out.print(temp.data+" ");
20             temp=temp.next;
21         }
22     }
23     public static void main(String[] args){
24         Main list=new Main();
25         list.insertBeginning(30);
26         list.insertBeginning(20);
27         list.insertBeginning(10);
28         list.display();
29     }
30 }
```

input

10 20 30

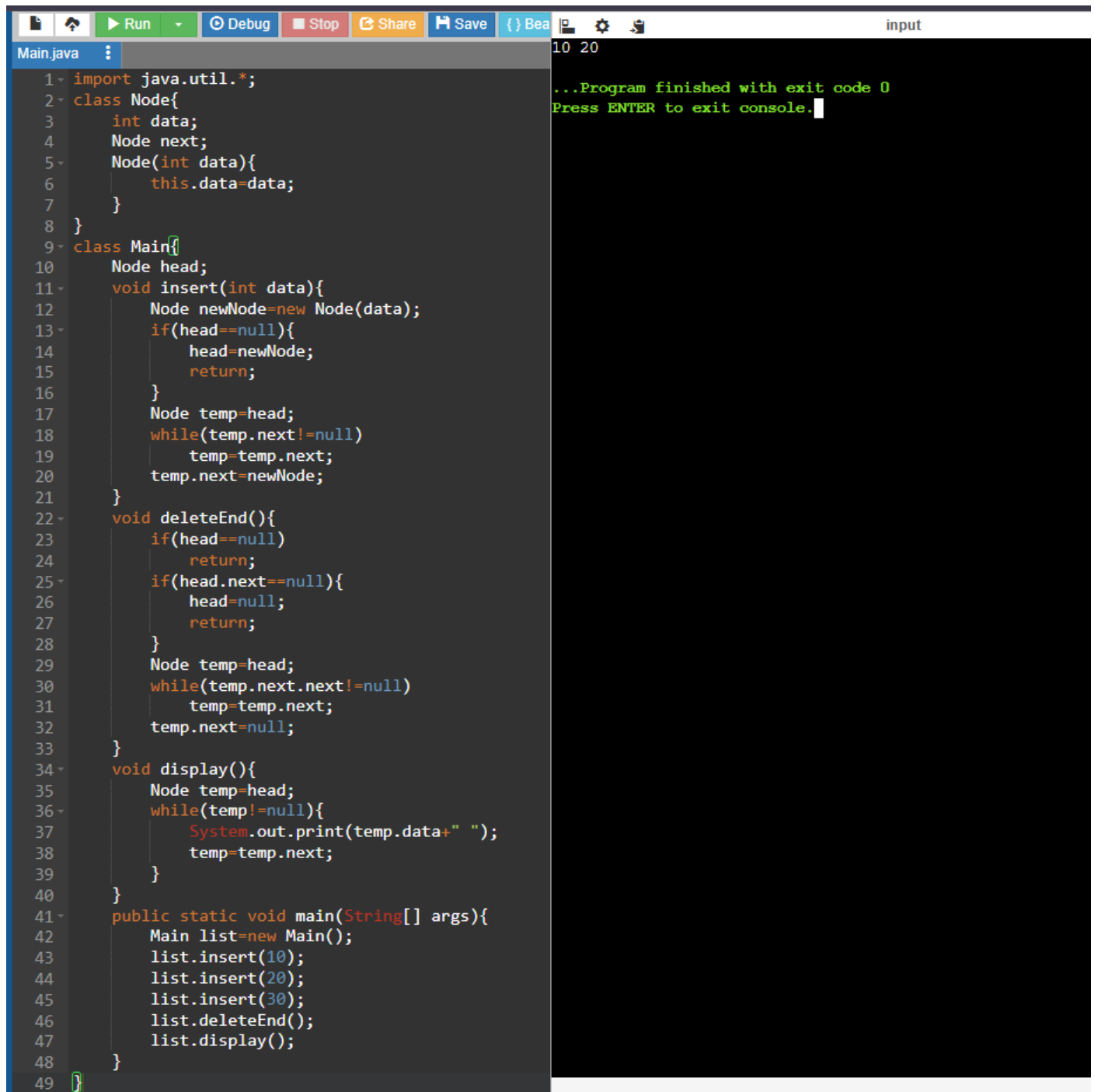
## 11. Delete at Beginning

```
Main.java 20 30
1 import java.util.*;
2 class Node{
3     int data;
4     Node next;
5     Node(int data){
6         this.data=data;
7     }
8 }
9 class Main{
10     Node head;
11     void insert(int data){
12         Node newNode=new Node(data);
13         if(head==null){
14             head=newNode;
15             return;
16         }
17         Node temp=head;
18         while(temp.next!=null)
19             temp=temp.next;
20         temp.next=newNode;
21     }
22     void deleteBeginning(){
23         if(head!=null)
24             head=head.next;
25     }
26     void display(){
27         Node temp=head;
28         while(temp!=null){
29             System.out.print(temp.data+" ");
30             temp=temp.next;
31         }
32     }
33     public static void main(String[] args){
34         Main list=new Main();
35         list.insert(10);
36         list.insert(20);
37         list.insert(30);
38         list.deleteBeginning();
39         list.display();
40     }
41 }
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```



## 12. Delete at End



The image shows a Java IDE with a code editor on the left and a console on the right. The code editor displays the following Java code:

```
1 import java.util.*;
2 class Node{
3     int data;
4     Node next;
5     Node(int data){
6         this.data=data;
7     }
8 }
9 class Main{
10     Node head;
11     void insert(int data){
12         Node newNode=new Node(data);
13         if(head==null){
14             head=newNode;
15             return;
16         }
17         Node temp=head;
18         while(temp.next!=null)
19             temp=temp.next;
20         temp.next=newNode;
21     }
22     void deleteEnd(){
23         if(head==null)
24             return;
25         if(head.next==null){
26             head=null;
27             return;
28         }
29         Node temp=head;
30         while(temp.next.next!=null)
31             temp=temp.next;
32         temp.next=null;
33     }
34     void display(){
35         Node temp=head;
36         while(temp!=null){
37             System.out.print(temp.data+" ");
38             temp=temp.next;
39         }
40     }
41     public static void main(String[] args){
42         Main list=new Main();
43         list.insert(10);
44         list.insert(20);
45         list.insert(30);
46         list.deleteEnd();
47         list.display();
48     }
49 }
```

The console on the right shows the output of the program:

```
10 20
...Program finished with exit code 0
Press ENTER to exit console.
```

### 13. Search in Linked List

```
Main.java  ⋮
1  import java.util.*;
2  class Node{
3      int data;
4      Node next;
5      Node(int data){
6          this.data=data;
7      }
8  }
9  class Main{
10     Node head;
11     void insert(int data){
12         Node newNode=new Node(data);
13         if(head==null){
14             head=newNode;
15             return;
16         }
17         Node temp=head;
18         while(temp.next!=null)
19             temp=temp.next;
20         temp.next=newNode;
21     }
22     void search(int key){
23         Node temp=head;
24         int pos=0;
25         while(temp!=null){
26             if(temp.data==key){
27                 System.out.println("Found at position 1");
28                 return;
29             }
30             pos++;
31             temp=temp.next;
32         }
33         System.out.println("Not Found");
34     }
35     public static void main(String[] args){
36         Main list=new Main();
37         list.insert(10);
38         list.insert(20);
39         list.insert(30);
40         list.search(20);
41     }
42 }
```

```
Found at position 1
...Program finished with exit code
Press ENTER to exit console.
```

## 14. Doubly Linked List (Insert + Display)

Main.java

```
1 import java.util.*;
2 class Node{
3     int data;
4     Node prev,next;
5     Node(int data){
6         this.data=data;
7     }
8 }
9 class Main{
10     Node head;
11     void insert(int data){
12         Node newNode=new Node(data);
13         if(head==null){
14             head=newNode;
15             return;
16         }
17         Node temp=head;
18         while(temp.next!=null)
19             temp=temp.next;
20         temp.next=newNode;
21         newNode.prev=temp;
22     }
23     void display(){
24         Node temp=head;
25         while(temp!=null){
26             System.out.print(temp.data+" ");
27             temp=temp.next;
28         }
29     }
30     public static void main(String[] args){
31         Main list=new Main();
32         list.insert(10);
33         list.insert(20);
34         list.insert(30);
35         list.display();
36     }
37 }
```

10 20 30

...Program finished with exit code 0  
Press ENTER to exit console.

## 15. Circular Singly Linked List (Insert + Display)

```
Main.java 1
1 import java.util.*;
2 class Node{
3     int data;
4     Node next;
5     Node(int data){
6         this.data=data;
7     }
8 }
9 class Main{
10     Node head;
11     void insert(int data){
12         Node newNode=new Node(data);
13         if(head==null){
14             head=newNode;
15             head.next=head;
16             return;
17         }
18         Node temp=head;
19         while(temp.next!=head)
20             temp=temp.next;
21         temp.next=newNode;
22         newNode.next=head;
23     }
24     void display(){
25         if(head==null)
26             return;
27         Node temp=head;
28         do{
29             System.out.print(temp.data+" ");
30             temp=temp.next;
31         }while(temp!=head);
32     }
33     public static void main(String[] args){
34         Main list=new Main();
35         list.insert(10);
36         list.insert(20);
37         list.insert(30);
38         list.display();
39     }
40 }
```

```
10 20 30
...Program finished with exit co
Press ENTER to exit console.
```

## 16. Circular Doubly Linked List (Insert + Display)

```
Main.java 10 20 30
1 import java.util.*;
2 class Node{
3     int data;
4     Node prev,next;
5     Node(int data){
6         this.data=data;
7     }
8 }
9 class Main{
10     Node head;
11     void insert(int data){
12         Node newNode=new Node(data);
13         if(head==null){
14             head=newNode;
15             head.next=head;
16             head.prev=head;
17             return;
18         }
19         Node last=head.prev;
20         last.next=newNode;
21         newNode.prev=last;
22         newNode.next=head;
23         head.prev=newNode;
24     }
25     void display(){
26         if(head==null)
27             return;
28         Node temp=head;
29         do{
30             System.out.print(temp.data+" ");
31             temp=temp.next;
32         }while(temp!=head);
33     }
34     public static void main(String[] args){
35         Main list=new Main();
36         list.insert(10);
37         list.insert(20);
38         list.insert(30);
39         list.display();
40     }
41 }
```

## 17. Recursive Function (Factorial)

```
Main.java 120
1 import java.util.*;
2 class Main{
3     static int fact(int n){
4         if(n==0||n==1)
5             return 1;
6         return n*fact(n-1);
7     }
8     public static void main(String[] args){
9         System.out.println(fact(5));
10    }
11 }
```

## 18. Recursive Binary Search

```
Main.java  ⋮
1  import java.util.*;
2  class Main{
3      static int binarySearch(int arr[],int low,int high,int key){
4          if(low>high)
5              return -1;
6          int mid=(low+high)/2;
7          if(arr[mid]==key)
8              return mid;
9          if(arr[mid]>key)
10             return binarySearch(arr,low,mid-1,key);
11             return binarySearch(arr,mid+1,high,key);
12     }
13     public static void main(String[] args){
14         int arr[]={10,20,30,40,50};
15         int index=binarySearch(arr,0,arr.length-1,40);
16         System.out.println(index);
17     }
18 }
```

input

3

## 19. Reverse an Array

```
Main.java  ⋮
1  import java.util.*;
2  class Main{
3      public static void main(String[] args){
4          int arr[]={1,2,3,4,5};
5          int i=0,j=arr.length-1;
6          while(i<j){
7              int temp=arr[i];
8              arr[i]=arr[j];
9              arr[j]=temp;
10             i++;
11             j--;
12         }
13         for(i=0;i<arr.length;i++)
14             System.out.print(arr[i]+" ");
15     }
16 }
```

input

5 4 3 2 1

## 20. Reverse a Linked List

```

Main.java
1  import java.util.*;
2  class Node{
3      int data;
4      Node next;
5      Node(int data){
6          this.data=data;
7      }
8  }
9  class Main{
10     Node head;
11     void insert(int data){
12         Node newNode=new Node(data);
13         if(head==null){
14             head=newNode;
15             return;
16         }
17         Node temp=head;
18         while(temp.next!=null)
19             temp=temp.next;
20         temp.next=newNode;
21     }
22     void reverse(){
23         Node prev=null;
24         Node curr=head;
25         while(curr!=null){
26             Node next=curr.next;
27             curr.next=prev;
28             prev=curr;
29             curr=next;
30         }
31         head=prev;
32     }
33     void display(){
34         Node temp=head;
35         while(temp!=null){
36             System.out.print(temp.data+" ");
37             temp=temp.next;
38         }
39     }
40     public static void main(String[] args){
41         Main list=new Main();
42         list.insert(10);
43         list.insert(20);
44         list.insert(30);
45         list.reverse();
46         list.display();
47     }
48 }

```

30 20 10

...Program finished with  
Press ENTER to exit console